

REQUIREMENT

=====

Build an employee leave application for our organization.

ARCHITECTURE

=====

4. ARCHITECTURE DESIGN

Architecture and Diagram Summary

- **Frontend Layer:** User interface for employees to submit leave requests and for HR personnel to approve them.
- **API Layer:** RESTful API endpoints for handling leave request operations.
- **Application Services Layer:** Business logic for processing leave requests, approvals, and leave balance calculations.
- **Persistence Layer:** Data storage and retrieval using Spring Data JPA and PostgreSQL.
- **Security Layer:** User authentication and authorization using Spring Security.
- **External Systems:** Integration with third-party services if needed (not in scope for this version).

Code Design

- `com.example.leave.web`: REST controllers and request/response DTOs.
- `com.example.leave.service`: Service classes for business logic.
- `com.example.leave.repository`: JPA repository interfaces.
- `com.example.leave.model`: Domain entities.
- `com.example.leave.security`: Security configuration and user details service.
- `com.example.leave.validation`: Custom validation annotations and validators.

Implementation Plan

Phase 1: Setup

Tasks:

- Initialize the project with Spring Boot.
- Set up PostgreSQL database.
- Configure Spring Security.

Dependencies: None

Expected Outcome: A working Spring Boot application with a secured entry point.

Phase 2: Database

Tasks:

- Design and create database schema.
- Implement JPA entities and repositories.

Dependencies: None

Expected Outcome: A database schema ready for application use.

Phase 3: Domain

Tasks:

- Define domain models for leave requests and users.
- Implement domain services.

Dependencies: Database schema

Expected Outcome: Domain models and services in place.

Phase 4: Business Logic

Tasks:

- Implement leave request submission logic.
- Implement leave approval workflow.
- Implement leave balance tracking.

Dependencies: Domain models and services

Expected Outcome: Complete business logic for leave management.

Phase 5: APIs

Tasks:

- Create REST API endpoints for leave requests.
- Implement request and response mapping.

Dependencies: Business logic

Expected Outcome: Functional RESTful APIs for leave management.

Phase 6: Testing and Deployment

Tasks:

- Write unit and integration tests.
- Set up continuous integration/continuous deployment (CI/CD).
- Deploy the application.

Dependencies: APIs

Expected Outcome: A tested and deployed application ready for use.

5. API DESIGN

Endpoints

M e t h o d	Path	P u r p o s e	Request Body	Response Body	S u c c e s s S t a t u s	E r r o r S t a t u s
P O S T	/leave -reque sts	S u b m i t a n e w l e a v e r e q u e s t	{"employeeId":"1", "leav eType":"SICK", "startDat e":"2023-10-01", "endDat e":"2023-10-05"}	{"id":"1", "status":"PENDING"}	2 0 1	4 0 0 4 0 4 0 9

M e t h o d	Path	P u r p o s e	Request Body	Response Body	S u c c e s s t a t u s	E r r o r S t a t u s
GET	/leave-requests/{id}	Retrieve leave request by ID	N/A	{"id": "1", "employeeId": "1", "leaveType": "SICK", "startDate": "2023-10-01", "endDate": "2023-10-05", "status": "PENDING"}	200	404

M e t h o d	Path	P u r p o s e	Request Body	Response Body	S u c c e s s t a t u s	E r r o r S t a t u s
GET	/leave-reque sts	R e t r i e v e a l l e a v e r e q u e s t s f o r a n e m p l o y e e	N/A	[{"id":"1","employeeId":"1","leaveType":"SICK","startDate":"2023-10-01","endDate":"2023-10-05","status":"PENDING"}]	200	404

M e t h o d	Path	P u r p o s e	Request Body	Response Body	S u c c e s s t a t u s	E r r o r S t a t u s
P A T C H	/leave-requests/{id}/approve	A p p r o v e a l l e a v e r e q u e s t	N/A	{"id":"1","status":"APPROVED"}	200	404

M e t h o d	Path	P u r p o s e	Request Body	Response Body	S u c c e s s t a t u s	E r r o r S t a t u s
P A T C H	/leave -reque sts/{id }/rejec t	R e j e c t a l e a v e r e q u e s t	N/A	{"id":"1","status":"REJECTED"}	2 0 0 4 0 4 0 9	4 0 0 4 0 4 0 9

M e t h o d	Path	P u r p o s e	Request Body	Response Body	S u c c e s s t a t u s	E r r o r S t a t u s
GET	/leave-balance/{employeeId}	Retrieve leave balance for an employee	N/A	{"employeeId": "1", "leaveType": "SICK", "balance": 10}	200	404

Validation and Error Handling

Field Level Validation Rules

- employeeId: Required, must be a valid employee ID.
- leaveType: Required, must be one of ["SICK", "VACATION", "PERSONAL"].
- startDate: Required, must be a valid date in YYYY-MM-DD format.
- endDate: Required, must be a valid date in YYYY-MM-DD format, and must be after startDate.

Standard Error Response Envelope

```
{
  "timestamp": "2023-10-01T12:34:56.789Z",
  "status": 400,
  "error": "Bad Request",
  "message": "Validation failed for object='leaveRequest'. Error count: 2",
  "errors": [
    {
      "object": "leaveRequest",
      "field": "startDate",
      "rejectedValue": "2023-10-05",
      "message": "must be after 2023-10-01"
    },
    {
      "object": "leaveRequest",
      "field": "leaveType",
      "rejectedValue": "HOLIDAY",
      "message": "must be one of [SICK, VACATION, PERSONAL]"
    }
  ]
}
```

Business Failures to HTTP Status Codes

Business Failure	HTTP Status Code
Invalid request data	400
Leave request not found	404
Conflicting leave request	409

DATA MODEL

=====

6. DATA MODEL

Model

Table	Column	Data Type	Nullable	Key	Default	Description
Employee	id	UUID	No	PK		Unique identifier for employee
Employee	name	VARCHAR	No			Employee's full name
Employee	email	VARCHAR	No			Employee's email address
LeaveRequest	id	UUID	No	PK		Unique identifier for leave request
LeaveRequest	employee_id	UUID	No	FK		Employee who requested leave
LeaveRequest	start_date	DATE	No			Start date of leave
LeaveRequest	end_date	DATE	No			End date of leave
LeaveRequest	status	VARCHAR	No		Pending	Status of leave request
LeaveRequest	created_at	TIMESTAMP	No			Timestamp when request was created
LeaveRequest	updated_at	TIMESTAMP	No			Timestamp when request was last updated

ER Diagram

```
{
  "entities": [
    { "name": "Employee", "fields": ["id", "name", "email"] },
    { "name": "LeaveRequest", "fields": ["id", "employee_id", "start_date", "end_date", "status"] },
  ],
  "relationships": [
    { "from": "Employee", "to": "LeaveRequest", "label": "1:N" }
  ]
}
```

Schema Script

```
CREATE TABLE Employee (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name VARCHAR NOT NULL,
  email VARCHAR NOT NULL UNIQUE
);

CREATE TABLE LeaveRequest (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  employee_id UUID NOT NULL,
  start_date DATE NOT NULL,
  end_date DATE NOT NULL,
  status VARCHAR NOT NULL DEFAULT 'Pending',
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (employee_id) REFERENCES Employee(id) ON DELETE CASCADE
);

CREATE INDEX idx_leave_request_employee_id ON LeaveRequest(employee_id);
CREATE INDEX idx_leave_request_status ON LeaveRequest(status);
```

Constraints and Indexes

- Primary Keys: `Employee.id`, `LeaveRequest.id`
- Foreign Key: `LeaveRequest.employee_id` references `Employee.id`
- Unique Constraint: `Employee.email`
- Check Constraint: `LeaveRequest.status` must be one of 'Pending', 'Approved', 'Rejected'
- Indexes:
 - `idx_leave_request_employee_id`: To speed up queries that filter or join on `employee_id`.
 - `idx_leave_request_status`: To speed up queries that filter on the status of leave requests.

DOCUMENTATION WITH REMAINING PLACEHOLDERS

=====

API Documentation

Overview

Base URL: `http://localhost:5000` (development) | `https://api.example.com` (production)

Response Format: JSON

Authentication: Bearer token

Rate Limits: 100 requests per minute

Authentication

Bearer Token

1. Obtain a token by logging in through the `/auth/login` endpoint with username and password.
2. Include in request headers: `Authorization: Bearer <token>`
3. Token expires in 1 hour; refresh using the `/auth/refresh` endpoint.

Example Request with Auth

```
curl -X GET http://localhost:5000/api/leave-requests \
-H "Authorization: Bearer your_token_here" \
-H "Content-Type: application/json"
```

Endpoints

POST /api/leave-requests

Submit a new leave request.

Request:

- **Method:** POST
- **Path:** `/api/leave-requests`
- **Headers:** `Authorization: Bearer <token>`, `Content-Type: application/json`
- **Request Body:**

```
{
  "employeeId": "1",
  "leaveType": "SICK",
  "startDate": "2023-10-01",
  "endDate": "2023-10-05"
}
```

Response:

- **Status 201 (Created):**

```
{
  "success": true,
  "data": {
    "id": "1",
    "status": "PENDING"
  },
  "message": "Leave request submitted successfully"
}
```

- **Status 400 (Bad Request):** Missing or invalid required field
- **Status 401 (Unauthorized):** Invalid or expired token
- **Status 403 (Forbidden):** Insufficient permissions
- **Status 409 (Conflict):** Conflicting leave request
- **Status 500 (Server Error):** Unexpected error

Example:

```
curl -X POST http://localhost:5000/api/leave-requests \
-H "Authorization: Bearer token" \
-H "Content-Type: application/json" \
-d '{"employeeId":"1","leaveType":"SICK","startDate":"2023-10-01","endDate":"2023-10-05"}
```

GET /api/leave-requests/{id}

Retrieve a leave request by ID.

Request:

- **Method:** GET
- **Path:** /api/leave-requests/{id}
- **Headers:** Authorization: Bearer <token>, Content-Type: application/json

Response:

- **Status 200 (OK):**

```
{
  "success": true,
  "data": {
    "id": "1",
    "employeeId": "1",
    "leaveType": "SICK",
    "startDate": "2023-10-01",
    "endDate": "2023-10-05",
    "status": "PENDING"
  },
  "message": "Leave request retrieved successfully"
}
```

- **Status 400 (Bad Request):** Missing or invalid required field

- **Status 401 (Unauthorized):** Invalid or expired token
- **Status 403 (Forbidden):** Insufficient permissions
- **Status 404 (Not Found):** Leave request not found
- **Status 500 (Server Error):** Unexpected error

Example:

```
curl -X GET http://localhost:5000/api/leave-requests/1 \  
  -H "Authorization: Bearer token" \  
  -H "Content-Type: application/json"
```

###